

- title: 智购 SmartShopping — 软件工程项目答辩slug: smartshopping-defensesummary: 基于 Spring MVC + GPT-4o-mini 的校园 B2C 购物平台，融合 AI 对话、语音识别、智能周报等多模态 AI 功能。答辩材料含技术架构、核心 AI 功能详解及 Q&A。
- 智购 SmartShopping — 软件工程项目答辩材料
 - 目录
 - 1. 项目概述
 - Q: 这个项目是做什么的?
 - Q: 项目的核心创新点是什么?
 - 2. 技术架构
 - Q: 项目用了哪些技术栈?
 - Q: 项目采用什么设计模式/架构?
 - Q: 前端如何与后端交互?
 - 3. 核心 AI 功能详解
 - 3.1 AI 对话购物系统
 - Q: AI 对话的完整调用链路是什么?
 - Q: System Prompt 是如何设计的?
 - Q: 如何防止商家误触发购物操作?
 - 3.2 语音识别系统
 - Q: 语音识别从录音到文字的完整流程是什么?
 - Q: 如何处理浏览器的兼容问题?
 - 3.3 AI 购物周报系统（新增核心亮点）
 - Q: AI 购物周报是什么?
 - Q: 周报的智能缓存机制是什么?
 - Q: AI 失败时的兜底策略是什么?
 - 4. 前端设计亮点
 - Q: 前端设计有何特色?
 - Q: 商品轮播是如何实现的?
 - 5. 项目 Q&A 问答
 - Q1: 为什么选择 Spring MVC 而不是 Spring Boot?
 - Q2: 为什么使用 Hibernate 而不是 MyBatis?
 - Q3: AI 对话的上下文窗口管理?
 - Q4: 系统安全性如何保障?
 - Q5: 项目扩展性如何?
 - Q6: AI 周报为什么要做缓存?
 - 6. 数据库设计
 - 核心表结构

- [订单状态流转](#)
- [品类映射](#)

title: 智购 SmartShopping — 软件工程项目答辩
slug: smartshopping-defense
summary: 基于 Spring MVC + GPT-4o-mini 的校园 B2C 购物平台，融合 AI 对话、语音识别、智能周报等多模态 AI 功能。答辩材料含技术架构、核心 AI 功能详解及 Q&A。

智购 SmartShopping — 软件工程项目答辩材料

校园宿舍一站式购物平台，集成 GPT-4o-mini 大语言模型、百度语音识别、AI 购物周报的 Java Web 应用

目录

- [1. 项目概述](#)
- [2. 技术架构](#)
- [3. 核心 AI 功能详解](#)
- [4. 前端设计亮点](#)
- [5. 项目 Q&A 问答](#)
- [6. 数据库设计](#)

1. 项目概述

Q: 这个项目是做什么的？

智购（SmartShopping）是一个面向大学校园宿舍楼场景的 **B2C 在线购物平台**，以华中农业大学荟园 99 栋为原型。区别于传统电商系统，项目深度融合了 **三项 AI 技术**：大语言模型对话、语音识别输入、AI 数据洞察。

服务于三类用户：

角色	功能
普通买家 (role=0)	浏览商品、搜索、加入购物车、下单、查看订单、发表评价、查看 AI 购物周报
商家 (role=1)	管理商品（上架/下架/编辑）、处理订单（发货）、查看销售记录、AI 经营建议
管理员	管理所有用户、管理帖子、开启/关闭发帖功能

Q: 项目的核心创新点是什么？

四大 AI 创新功能，覆盖购物全链路：

- 1. **AI 对话购物** — 接入 GPT-4o-mini 大语言模型，用户用自然语言与 AI 对话即可完成商品推荐、加入购物车
- 2. **语音识别输入** — 集成百度短语音识别 API，按住麦克风说话自动转文字发送给 AI，实现无键盘购物
- 3. **双角色 AI 系统** — 顾客模式下 AI 充当「购物小助手」，商家模式下 AI 充当「店铺管理助手」，前端界面和后端 System Prompt 均按角色切换
- 4. **AI 购物周报** — 每周自动分析用户的购物数据，生成个性化的消费画像报告（Spotify Wrapped 风格），包含健康提醒、趣味头衔、AI 推荐商品

2. 技术架构

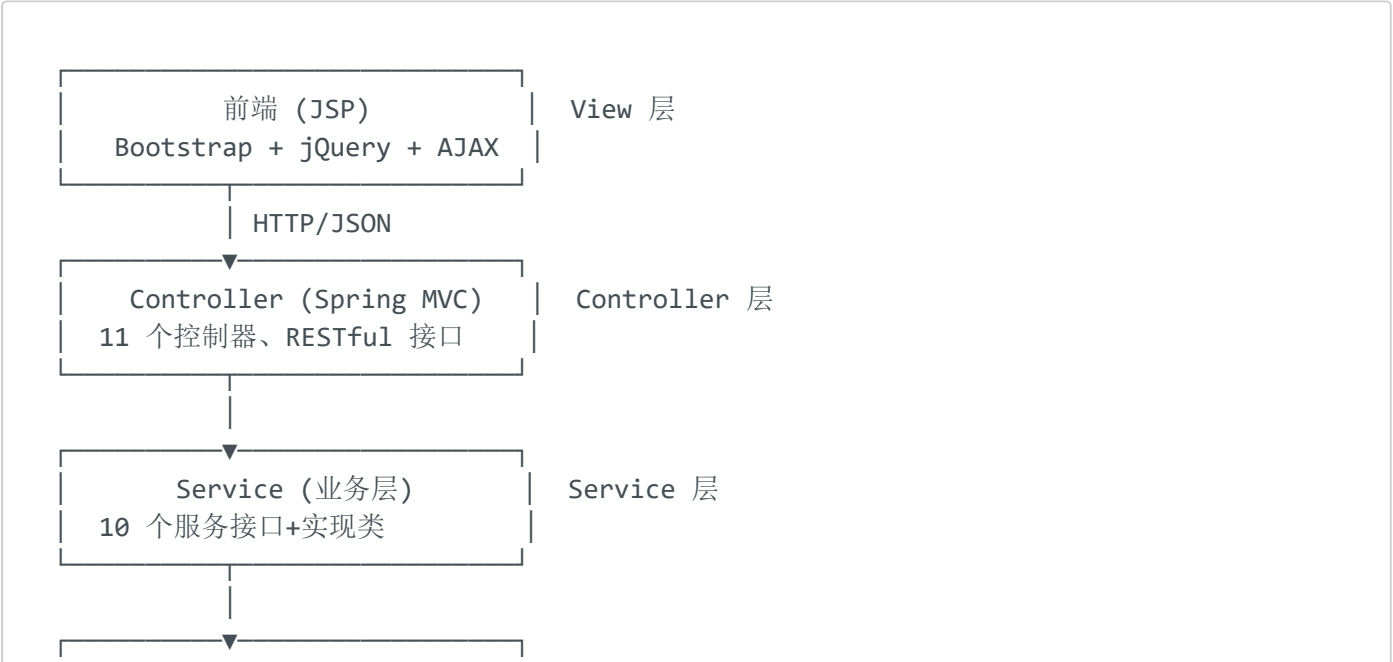
Q: 项目用了哪些技术栈？

层级	技术	版本	用途
前端	JSP + JSTL + Bootstrap 3	—	服务端渲染 + AJAX 动态交互
前端 JS	jQuery + Layer.js	—	AJAX 请求、弹窗、加载动画

层级	技术	版本	用途
后端框架	Spring MVC	4.3.5	DispatcherServlet + RESTful 接口
ORM	Hibernate	4.3.8	DAO 层实体映射、自动建表
数据库	MySQL	8.0.13	关系型存储，支持事务
构建工具	Maven	3.x	依赖管理、编译打包
服务器	Apache Tomcat	11.0	Servlet 容器
编程语言	Java	1.8	—
AI 大模型	OpenAI GPT-4o-mini	—	通过 REST API + System Prompt 调用
语音识别	百度 AI 短语音识别	—	OAuth2 鉴权 + RAW 模式 WAV 上传
HTTP 客户端	Apache HttpClient	4.5.13	AI API 和语音 API 的 HTTP 请求
JSON 处理	FastJSON 1.2.24 + Jackson 2.9	—	序列化/反序列化
前端音频	Web Audio API	—	PCM 采集、降采样、WAV 编码

Q: 项目采用什么设计模式/架构?

经典三层架构 + MVC 模式:



关键设计模式：

- **IoC/DI**：通过 Spring `@Resource` 注解实现依赖注入
- **策略模式**：AI System Prompt 根据 `role` 参数动态切换
- **Session 管理**：Hibernate Session 由 Spring 的 `OpenSessionInViewFilter` 管理
- **AOP**：使用 Spring Aspects 进行横切关注点管理
- **缓存模式**：AI 周报使用 Session 缓存 + 哈希校验，避免重复调用 AI

Q: 前端如何与后端交互？

前端通过 jQuery AJAX 发送 POST 请求到 Spring MVC 控制器，控制器返回 JSON (`@ResponseBody`)，前端动态生成 HTML 渲染页面。

```
// 示例：AI 对话请求
$.ajax({
    type: 'POST',
    url: '${cp}/aiChat',
    data: { message: text, historyJson: JSON.stringify(aiChatHistory) },
    success: function(result) { /* 渲染 AI 回复 */ }
});
```

3. 核心 AI 功能详解

3.1 AI 对话购物系统

Q: AI 对话的完整调用链路是什么？

```
用户输入文字
|
▼
前端 main.jsp - POST /aiChat {message, historyJson}
|
▼
AiController.aiChat()
|  1. 从 Session 获取 currentUser.role (0=顾客/1=商家)
|  2. 解析 client 传来的对话历史 JSON
|  3. 调用 aiService.chat(message, history, role)
```

▼
AiServiceImpl.chat()
| 1. 查询全量商品列表作为上下文
| 2. 根据 role 构造不同的 System Prompt
| 3. 组装消息列表: system + history(近20条) + user
| 4. POST 到 GPT-4o-mini API (OpenAI 兼容格式)
| 5. 返回 AI 生成的文本

▼
AiController.aiChat()
| 解析 AI 返回的 JSON 指令:
| - "recommend_product" → 返回商品推荐卡片
| - "add_to_cart" → 自动写入购物车表
| - 其他 → 返回自然语言回复

▼
前端渲染结果 (打字机动画 / 商品卡片 / 加入购物车按钮)

Q: System Prompt 是如何设计的?

顾客模式 Prompt 核心要点:

- 动态注入完整商品清单作为上下文
- 要求 AI 在特定场景输出 **结构化 JSON**:
`{"action": "recommend_product", "product_name": "..."} 或`
`{"action": "add_to_cart", "product_name": "...", "count": 1}`
- 强调上下文推断: 用户说"帮我加到购物车"而未提商品名时, 结合对话历史推断
- 主动推荐: 模糊需求 (如"想喝水") 直接推荐而不反问
- JSON 不含 Markdown 标记, 精确匹配商品名

商家模式 Prompt 核心要点:

- 称呼用户为「老板」
- 引导商家使用左侧导航栏功能 (处理订单、商品管理)
- 主动提供经营建议
- **禁止**输出购物相关 JSON 指令

Q: 如何防止商家误触发购物操作?

双重防护:

1. **Prompt 层面**: 商家模式下 System Prompt 明确禁止推荐商品给商家购买
2. **Controller 层面**: 即使 AI 输出了购物 JSON, **AiController** 检查 `role == 1` 时直接拦截, 返回提示「老板, 这是顾客功能哦~」

3.2 语音识别系统

Q: 语音识别从录音到文字的完整流程是什么？



Q: 如何处理浏览器的兼容问题？

- **AudioContext 兼容：**`window.AudioContext || window.webkitAudioContext`
- **零增益防回授：**`scriptProcessor → GainNode(gain=0) → destination`
- **降采样：**线性插值从 44.1k/48k 到 16kHz
- **WAV 编码：**完整写入 RIFF 头（采样率、位深、声道数）
- **错误分类：**区分 `NotAllowedError`(权限拒绝)、`NotFoundError`(无设备)、`NotReadableError`(设备占用)，给出中文提示

3.3 AI 购物周报系统（新增核心亮点）

Q: AI 购物周报是什么？

每周自动分析用户购物数据，生成个性化消费报告。模仿 **Spotify Wrapped** 的视觉风格，包含：

模块	内容
统计卡片	购买总量、总花费、最爱品类、下单高峰时段

模块	内容
趣味称号	AI 生成的个性化头衔（「深夜觅食者」「零食收藏家」）
日期范围	本周周一~周日的具体日期（如 05/26 - 06/01）
AI 点评	基于购物数据的一对一幽默评论
健康提醒	根据购买模式生成（深夜零食→「给肠胃放个假」、速食多→「搭配蔬菜水果」）
智能推荐	AI 推荐的 3 个下周商品

Q: 周报的智能缓存机制是什么？

为避免重复调用 AI（成本 + 响应时间），设计了 **Session 级别缓存**：



缓存失效条件：用户产生新购物记录 → hash 变化 → 下次请求自动重新生成。

Q: AI 失败时的兜底策略是什么？

即使 AI 完全不可用，系统仍能从原始购物数据生成统计报告：

- 从品类分布、时段分布、日期分布直接计算统计值
- 使用预设称号库按用户 ID 取模分配
- 生成基于数据的通用点评
- 健康提醒按购买模式匹配预设规则

保证用户始终能看到周报，不会因为 AI 异常出现空白。

4. 前端设计亮点

Q: 前端设计有何特色？

完整设计演进经历了三次迭代，最终采用 **Vintage Campus（复古校园）** 美学：

设计要素	具体实现
色板	暖纸白 #f6f3ed + 深棕墨色 #2c2416 + 青绿 #1b6b6b + 砖红 #c44536
纹理	SVG 噪点叠加 <code>body::after</code> ，模拟纸质感
字体	Segoe UI / PingFang SC / Hiragino Sans GB 多层次回退
商品轮播	60fps <code>requestAnimationFrame</code> 连续流动 + 渐变遮罩 + 半卡边缘
AI 面板	右侧书脊抽屉式滑出，不覆盖导航栏
购物周报	深色主题 + 暗绿渐变卡片 + 脉动骨架屏加载

Q: 商品轮播是如何实现的？

- **60fps 连续动画**：`requestAnimationFrame` 每帧 `scrollLeft += speed`，摒弃传统 `setInterval` 逐张跳跃
- **左右渐变遮罩**：`::before/::after` 伪元素从背景色渐变到透明，卡片进出边缘时自然淡入淡出
- **半卡边缘**：Track 内边距 60px，首尾卡片始终露半截，提示用户有更多内容
- **无限循环**：克隆卡片填满 2x 容器宽度，`scrollLeft` 越过原始宽度时瞬间归零，裸眼无感知
- **错落有致**：偶数行右流、奇数行左流，各行初始偏移不同

5. 项目 Q&A 问答

Q1: 为什么选择 Spring MVC 而不是 Spring Boot？

Spring MVC + XML 配置方式有助于深入理解 Spring 框架核心概念（DispatcherServlet、依赖注入、AOP），而非 Spring Boot 的自动配置黑盒。项目规

模适中，Spring MVC 完全满足需求。

Q2: 为什么使用 Hibernate 而不是 MyBatis?

Hibernate 的 ORM 自动映射减少了手写 SQL 的工作量。通过 `@Entity` 注解定义实体，Hibernate 自动生成和维护表结构（`hibernate.hbm2ddl.auto=update`），提高了开发效率。

Q3: AI 对话的上下文窗口管理?

每次请求携带最近 20 条历史消息（约 10 轮对话），避免 token 超限。对话历史通过客户端 JavaScript 缓存（`aiChatHistory` 数组）+ 服务端 Session 双重备份，确保页面刷新后历史不丢失。

Q4: 系统安全性如何保障?

- **会话管理**：基于 HttpSession 的用户认证，未登录用户无法执行购物操作
- **角色隔离**：商家 AI 助手禁止触发购物操作，Controller 层面双重校验
- **SQL 注入**：Hibernate 参数化查询防止注入
- **密码存储**：通过 SHA256 + Salt 哈希存储

Q5: 项目扩展性如何?

AI 模块采用 **策略模式** 设计，通过 `role` 参数切换 System Prompt。新增角色只需：

1. 在 `AiServiceImplement.chat()` 中添加新的 `else if (role == N)` 分支
2. 编写对应的 System Prompt
3. 在 Controller 中添加对应的指令拦截逻辑

Q6: AI 周报为什么要做缓存?

每次周报生成需要调用一次 GPT-4o-mini API，响应时间 2-5 秒且有成本。缓存机制确保：

- 购物记录无变化时不重复调用
- 每周最多重新生成 3 次

- AI 故障时有兜底数据

6. 数据库设计

核心表结构

表名	说明	关键字段
product	商品表	id, name, price, img, type(1-7), bossId, store
user	用户表	id, name, email, password, role(0/1/?)
shoppingcar	购物车	userId, productId, counts, productPrice
shoppingrecord	订单记录	userId, productId, shopId, time, orderStatus(0/1/2), productPrice, counts
card	论坛帖子	id, userId, content
evaluation	商品评价	userId, productId, content
poststatus	发帖开关	id, status(0/1)

订单状态流转

未发货 (0) → 运输中 (1) → 已收货 (2)

- 买家下单创建 orderStatus=0
- 商家在「处理订单」页面点击发货 → orderStatus=1
- 买家在「我的订单」页面点击确认收货 → orderStatus=2

品类映射

type	品类
1	休闲零食
2	酒水饮料
3	方便速食
4	新鲜水果
5	日用百货
6	文具办公
7	其他

项目地址：<https://github.com/your-username/SmartShopping> **开发环境：**
IntelliJ IDEA + Tomcat 11 + MySQL 8.0 + JDK 1.8